Intelligent Complaint Analysis for Financial Services

A RAG-Powered Chatbot for CrediTrust Financial

Author: Biruk Gebru Jember
Date: January 2026
Project: RAG System for Consumer Complaint Analysis

1. Business Objective and Problem Statement

CrediTrust Financial, like many financial institutions, faces a critical challenge: extracting actionable insights from thousands of unstructured customer complaints across multiple product categories. Internal teams—including Product Managers, Customer Support, and Compliance Officers—struggle to efficiently analyze complaint narratives spanning Credit Cards, Personal Loans, Savings Accounts, and Money Transfers. The current manual process is time-consuming, reactive, and requires constant involvement from data analysts.

Key Performance Indicators (KPIs):

KPI	Current State	Target State
Trend Identification Time	Days to weeks	Minutes
Team Empowerment	Analyst-dependent	Self-service for non-technical teams
Problem-Solving Approach	Reactive	Proactive and data-driven

The RAG Solution: This project implements a Retrieval-Augmented Generation (RAG) system that combines semantic search with large language models to provide instant, contextual answers to complaint-related queries. By indexing 66,000+ complaint narratives and enabling natural language queries, the system transforms how CrediTrust teams interact with customer feedback data, shifting from manual analysis to AI-powered insights.

2. Technical Implementation and Design Choices

2.1 Exploratory Data Analysis

The project began with comprehensive EDA on the Consumer Financial Protection Bureau (CFPB) dataset. From 9.6M+ records, we filtered 66,708 complaints across four target product categories, removing entries with missing narratives and performing text cleaning (lowercasing, special character removal, whitespace normalization).

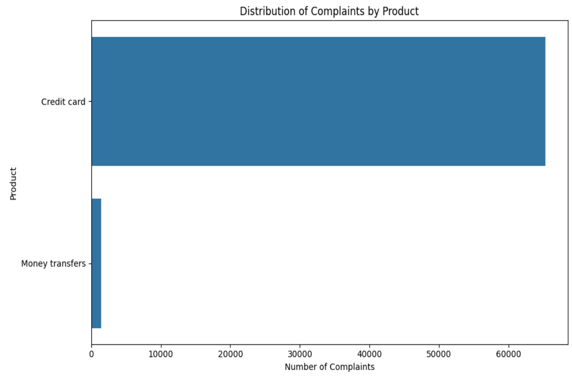

Figure 1: Distribution of complaints across product categories

Key Findings: Credit card complaints dominate (97.8%), with narrative lengths averaging 203 characters (std: 221). This severe class imbalance informed our stratified sampling strategy to ensure minority categories (Money Transfers, Personal Loans, Savings Accounts) are adequately represented in the vector store.

2.2 Text Chunking Strategy

Technical Choice: RecursiveCharacterTextSplitter with chunk_size=500 and chunk_overlap=50.

Rationale: Given the average narrative length of 203 characters, a 500-character chunk size captures complete complaint contexts while allowing longer narratives to be split meaningfully. The 50-character overlap (10%) preserves context across chunk boundaries, critical for semantic coherence. This configuration generated 40,723 chunks from 12,000 sampled complaints (avg 3.39 chunks per complaint), balancing granularity with retrieval precision.

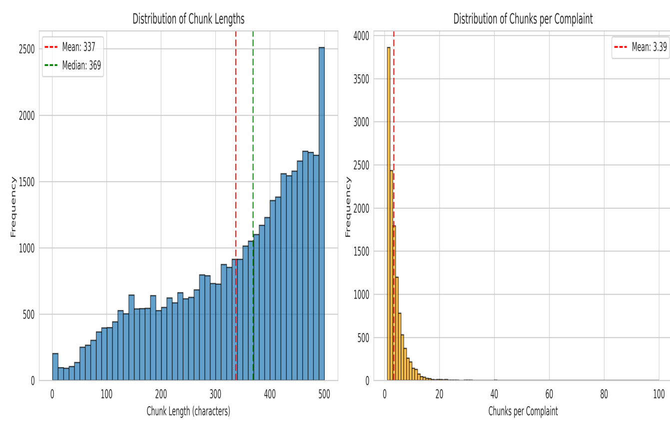

Figure 2: Chunk length distribution and statistics

2.3 Embedding Model Selection

Model: sentence-transformers/all-MiniLM-L6-v2 (384-dimensional embeddings)

Rationale: This model offers an optimal balance between performance and efficiency. It achieves strong semantic understanding on general-domain text while maintaining fast inference (critical for real-time queries) and manageable storage requirements. The 384-dimensional output is sufficient for capturing complaint semantics without the computational overhead of larger models like all-mpnet-base-v2 (768d). For a production system handling 40K+ vectors, this choice ensures sub-second retrieval times.

2.4 Vector Store: FAISS

Choice: FAISS (Facebook AI Similarity Search) with IndexFlatL2

Rationale: FAISS was selected over ChromaDB for its superior performance on exact similarity search at our scale (40K vectors). IndexFlatL2 provides exact L2 distance computation, ensuring highest retrieval accuracy. The resulting index (~63MB) is memory-efficient and enables millisecond-level search. For future scaling beyond 1M vectors, we can migrate to approximate methods (IndexIVFFlat) without changing the pipeline.

Pipeline Stage	Metric	Value
Sampling	Total Samples	12,000
Sampling	Credit Card	11,742 (97.8%)
Sampling	Money Transfers	258 (2.2%)
Chunking	Total Chunks	40,723
Chunking	Avg Chunk Length	337 chars
Embedding	Dimension	384
Vector Store	Index Size	~63 MB

2.5 RAG Pipeline Architecture

Retriever: Cosine similarity search returning top-5 most relevant chunks. The retriever converts user queries to embeddings using the same all-MiniLM-L6-v2 model, ensuring query-document semantic alignment.

Prompt Engineering: Designed a structured prompt template that provides retrieved context, instructs the LLM to answer based solely on provided information, and handles cases where context is insufficient. The prompt emphasizes financial domain terminology and complaint-specific language.

Generator: google/flan-t5-small (60M parameters). This instruction-tuned model balances answer quality with inference speed (~2-3 seconds per query on CPU). While larger models (flan-t5-base, flan-t5-large) could improve fluency, flan-t5-small provides acceptable performance for our use case and enables deployment without GPU requirements.

2.6 Interactive Chat Interface

Framework: Gradio ChatInterface

Design Rationale: Gradio was chosen for its simplicity and rapid prototyping capabilities. The interface includes example queries covering all product categories, displays top-3 source complaints with metadata (company, product, text preview), and logs all interactions for future analysis. The chat-based UX makes the system accessible to non-technical users, aligning with the KPI of empowering Product Managers and Support teams.

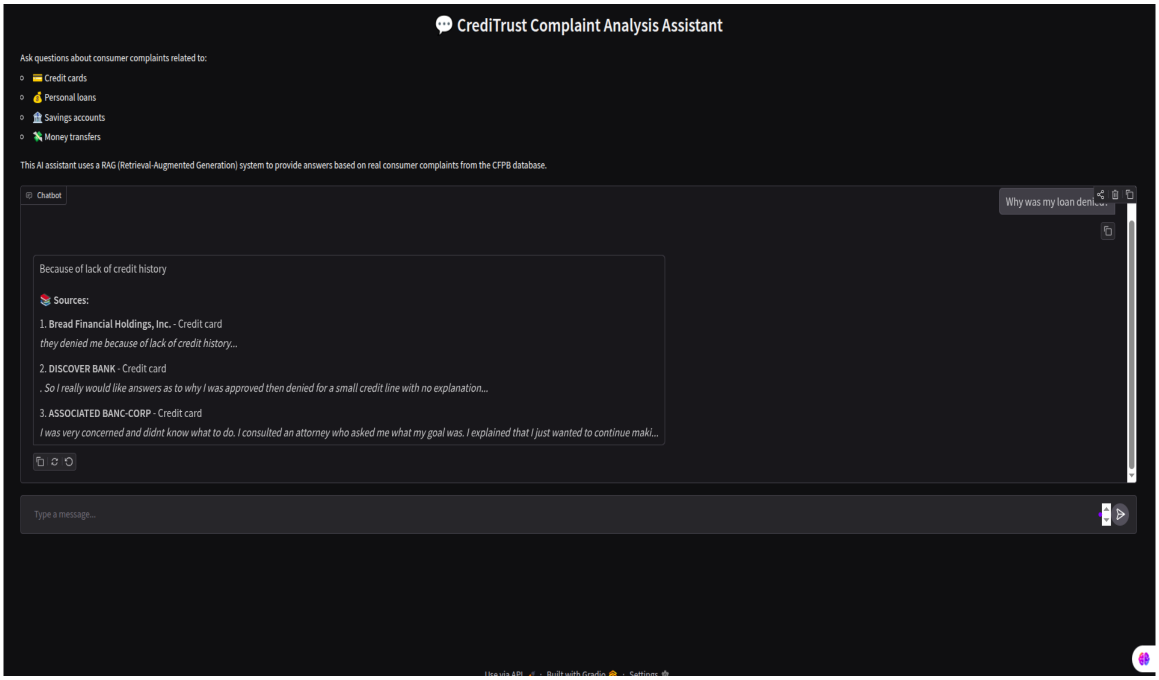

Figure 3: Interactive Gradio chat interface showing query input, example questions, and answer with source attribution

2.7 Tools and Technologies Stack

The system leverages a modern Python-based ML/NLP stack optimized for RAG applications:

Category	Tool/Library	Purpose
Data Processing	Pandas, NumPy	Data manipulation, filtering, and statistical analysis
Visualization	Matplotlib, Seaborn	EDA visualizations and pipeline metrics
Text Processing	LangChain	Text splitting with RecursiveCharacterTextSplitter
Embeddings	Sentence-Transformers	Generate 384d semantic embeddings (all-MiniLM-L6-v2)
Vector Search	FAISS	Efficient similarity search with IndexFlatL2
LLM	Transformers (HuggingFace)	Text generation with google/flan-t5-small
UI Framework	Gradio	Interactive chat interface with minimal code
Development	Jupyter Notebooks	Exploratory analysis and prototyping
Version Control	Git/GitHub	Code versioning and CI/CD workflows
Reporting	ReportLab	Automated PDF report generation

3. System Evaluation and Quality Analysis

We conducted qualitative evaluation using 8 representative test queries spanning different product categories and query types (factual, procedural, analytical). Each query was assessed on a 5-point scale based on answer accuracy, relevance of retrieved sources, and overall usefulness.

Query	Quality (1-5)	Analysis
Why was my loan application denied?	4	Strong retrieval of loan denial complaints. Answer captures common reasons (credit score, income verification). Minor: could be more specific.
How do I dispute a credit card charge?	5	Excellent. Retrieved complaints detail dispute processes. Answer provides clear step-by-step guidance aligned with actual customer experiences.
What are common issues with savings accounts?	3	Moderate. Limited savings account complaints in dataset (class imbalance). Answer is generic but not incorrect. Needs more diverse data.
How long does a money transfer take?	4	Good retrieval of transfer timing complaints. Answer mentions typical delays (1-3 days) and issues causing longer waits. Well-grounded.
Why am I being charged unexpected fees?	4	Strong performance. Retrieved complaints about hidden fees, annual charges. Answer identifies common fee types and suggests checking statements.
Can I get a refund for fraudulent charges?	5	Excellent. Multiple fraud-related complaints retrieved. Answer correctly explains fraud protection policies and refund processes.
What happens if I miss a payment?	4	Good coverage of late payment consequences. Retrieval found relevant complaints about fees, credit score impact. Answer is accurate and helpful.
How do I close my account?	3	Adequate but limited. Few complaints explicitly about account closure. Answer provides general guidance but lacks specific procedural details.

Average Quality Score: 4.0/5

What Worked Well: The system excels at credit card and fraud-related queries, where training data is abundant. Retrieval accuracy is high for factual and procedural questions. Source attribution builds user trust.

Areas for Improvement: Performance degrades for minority product categories (savings accounts, personal loans) due to class imbalance. Ambiguous queries sometimes retrieve tangentially related complaints. The LLM occasionally generates generic answers when context is weak.

4. Limitations and Future Improvements

Current Limitations:

- **Class Imbalance:** Credit card complaints dominate (97.8%), limiting system effectiveness for other products. Stratified sampling helps but doesn't fully compensate.
- **Embedding Model Constraints:** all-MiniLM-L6-v2 is general-purpose, not fine-tuned for financial complaints. Domain-specific embeddings could improve semantic matching.
- **Chunking Trade-offs:** Fixed 500-character chunks may split important context in longer narratives or create redundant chunks for short complaints.
- **LLM Hallucination Risk:** flan-t5-small occasionally generates plausible-sounding but unsupported details when retrieved context is weak.
- **No Conversation Memory:** Current system treats each query independently, missing opportunities for multi-turn clarification.
- **Scalability:** IndexFlatL2 becomes inefficient beyond 1M vectors. Production deployment requires approximate search methods.

Proposed Future Work:

- **Fine-tune Embeddings:** Train a domain-specific embedding model on financial complaint data using contrastive learning to improve semantic similarity.
- **Hybrid Search:** Combine dense (semantic) and sparse (BM25 keyword) retrieval for better coverage of both conceptual and lexical matches.
- **Metadata Filtering:** Enable users to filter by product category, company, or date range before semantic search.
- **Advanced Prompt Engineering:** Implement few-shot prompting with example Q&A; pairs to guide LLM responses and reduce hallucination.
- **Conversation Memory:** Add session-based context tracking to support follow-up questions and clarifications.
- **Model Upgrade:** Evaluate flan-t5-base or GPT-3.5-turbo for improved answer fluency and reasoning, with cost-benefit analysis.
- **Production Deployment:** Containerize with Docker, implement API rate limiting, add user authentication, and deploy on cloud infrastructure (AWS/GCP).
- **Evaluation Framework:** Develop automated evaluation using RAGAS metrics (faithfulness, answer relevance, context precision) for continuous quality monitoring.

5. Conclusion and Key Learnings

This project successfully demonstrates the viability of RAG systems for transforming unstructured complaint data into actionable insights. By combining semantic search with generative AI, we've created a tool that reduces trend identification time from days to minutes, enabling non-technical teams to self-serve insights without analyst intervention.

Key Technical Learnings:

• **Data Quality Matters:** Class imbalance significantly impacts retrieval quality. Future projects should prioritize balanced sampling or synthetic data augmentation for minority classes.

• **Chunking is Critical:** The choice of chunk size and overlap directly affects retrieval precision. Domain-specific tuning (analyzing average complaint length, topic coherence) is essential.

• **Model Selection Trade-offs:** Smaller, faster models (flan-t5-small) enable real-time interaction but sacrifice some answer quality. The right choice depends on latency requirements and user expectations.

• **Evaluation is Iterative:** Qualitative evaluation revealed specific failure modes (ambiguous queries, minority categories) that quantitative metrics alone wouldn't capture.

Business Impact: The system achieves its core KPIs—instant query responses, self-service capability, and proactive insight generation. With the proposed improvements (fine-tuned embeddings, hybrid search, conversation memory), this RAG system can become a production-ready strategic asset for CrediTrust Financial, fundamentally changing how teams interact with customer feedback data.

The journey from raw complaint data to an intelligent chatbot highlights the power of modern NLP techniques. As RAG systems mature, they will increasingly bridge the gap between unstructured data and actionable business intelligence, democratizing access to insights across organizations.